Факультет компьютерных технологий и прикладной математики

Кафедра анализа данных и искусственного интеллекта

09.03.03

Современные среды разработки интерактивных приложений

Лабораторная работа № 2

Каталог и база результатов заездов

1. Цель работы

Продолжить свой журнал заездов из ЛР 1. Представить записи объектами, показать их в таблице, добавить поиск и фильтр, сохранить данные между запусками с помощью SQLite. Реализовать чтение, добавление, изменение и удаление записей, сохранение копий JSON и XML и восстановление всего журнала без потери прежних данных при ошибке.

Работа рассчитана на два занятия по два академических часа с самостоятельной подготовкой. На первом занятии выполните этапы 1-5, на втором 6-9. Создайте свою форму и обработчики, используя уже выполненную ЛР 1. Эталонная реализация не является исходным проектом студента.

Среда: Windows, Visual Studio 2026, .NET 10 LTS и C#. Единственная добавляемая зависимость для этой работы: Microsoft.Data.Sqlite 10.0.11. Используйте современный шаблон Windows Forms App, а не вариант .NET Framework. Основные окна и создание обработчиков разобраны в ЛР 1 со встроенными снимками среды.

На снимках показаны команды и свойства Visual Studio 2022. Работу выполняйте в Visual Studio 2026, используя имена и настройки из задания.

2. Необходимая теория

Запись, свойство и объект

В ЛР 1 заезд представлялся одной строкой. Теперь нужен отдельный объект RaceResult со свойствами Id, Driver, Car, Track и LapSeconds. Класс описывает устройство записи, а оператор new создаёт конкретный объект.

Public означает доступность другим частям приложения. Свойство с get и set можно читать и изменять. Для Id используйте long, поскольку это целый идентификатор SQLite. Для времени достаточно int. Строковые свойства имеют тип string. Начальная пустая строка помогает создать объект, но перед сохранением обязательные значения всё равно проверяются.

Список, массив и цикл

List<RaceResult> означает изменяемый список объектов RaceResult. Add добавляет элемент, Count сообщает количество. Индекс в квадратных скобках является позицией в текущем списке и начинается с нуля. Он не заменяет Id. Конструкция foreach (RaceResult row in rows) выполняет тело для каждого объекта. Массив string[] подходит для заранее известных вариантов автомобиля и трассы. В этой работе эти варианты фиксированы, а журнал может расти.

Метод, параметр и возвращаемое значение

Метод получает параметры в круглых скобках. В записи List<RaceResult> GetAll() тип перед именем означает, что метод возвращает список. Void означает отсутствие возвращаемого значения. Return заканчивает вызов и при необходимости возвращает результат. Private ограничивает доступ текущим классом, public разрешает обращение из других классов. Static связывает метод с классом: RaceRules.Validate(row) вызывается без создания объекта RaceRules. Const используется для неизменных границ времени. Readonly у поля запрещает заменить его ссылку после инициализации, но не запрещает менять содержимое самого объекта.

Привязка данных

BindingSource связывает коллекцию с элементом управления. Сначала список назначают свойству DataSource у BindingSource, затем сам BindingSource назначают свойству DataSource таблицы. Таблица показывает объекты, но не сохраняет их в SQLite автоматически. HeaderText задаёт видимый заголовок столбца, DataPropertyName связывает его с точным

именем публичного свойства. При обычном List<T> после изменения содержимого может потребоваться ResetBindings(false). False означает, что устройство свойств не менялось. При назначении нового списка в DataSource отдельный вызов обычно не требуется.

Поиск, лямбда и LINQ

Лямбда r => r.LapSeconds < 90 является короткой функцией: она получает запись и возвращает логический результат. Здесь 90 означает конкретный запрос «быстрее полутора минут», а не новую границу допустимого времени. Метод Where оставляет объекты, для которых функция вернула true. OrderBy задаёт порядок, ToList выполняет запрос и создаёт список. Метод Contains с StringComparison.OrdinalIgnoreCase ищет подстроку без учёта регистра. Два последовательных Where означают, что должны пройти оба условия. Обычный List<RaceResult> не поддерживает строковую фильтрацию через BindingSource.Filter. Для него формируйте новый список с LINQ.

Ошибки и освобождение ресурсов

Throw передаёт исключение вызывающему коду и прекращает обычное выполнение метода. В обработчике кнопки блок try содержит операцию, а catch показывает причину отказа. Не оставляйте пустой catch и не выводите сообщение об успехе после отказа. Запись using Microsoft.Data.Sqlite подключает короткие имена типов из пространства имён. Другая конструкция, using SqliteConnection connection =.., освобождает соединение при выходе из области, в том числе при исключении. Аналогично закрываются потоки записи. Она не удаляет сохранённые данные.

SQLite и четыре операции

SQLite хранит записи между запусками. Microsoft.Data.Sqlite предоставляет классы соединения, команды и читателя. SELECT читает данные. INSERT добавляет запись. UPDATE меняет выбранную запись. DELETE удаляет её. У изменения и удаления обязательно укажите WHERE

Id=$id. Значение параметра передайте через Parameters.AddWithValue. Пользовательский текст не склеивается с SQL. ExecuteReader получает строки, ExecuteScalar одно значение, ExecuteNonQuery выполняет изменение и возвращает число затронутых строк. У Microsoft.Data.Sqlite нет полной реализации DbDataAdapter, поэтому в работе используется чтение объектов и явные команды.

Сериализация и восстановление

Сериализация превращает список объектов в представление JSON или XML. Десериализация читает его обратно. System.Text.Json работает с публичными свойствами. Для XmlSerializer используется публичный класс с доступным конструктором без параметров и публичными свойствами. Сохраняйте список RaceResult, а не форму или DataGridView. Typeof(List<RaceResult>) передаёт описание типа XML-сериализатору. Вопросительный знак в List<RaceResult>? предупреждает, что результата может не быть. Оператор as пытается получить объект указанного типа, при неудаче возвращает null. Null нужно проверить до обращения к данным.

Repository, транзакция и Unit of Work

RaceRepository предоставляет понятные операции GetAll, Add, Update, Delete и ReplaceAll. SQL находится в нём, а не в обработчиках формы. Транзакция объединяет удаление прежних строк и вставку новых в одну атомарную операцию. Commit выполняется только после успеха всех команд. Незавершённая транзакция откатывается при освобождении. Unit of Work в архитектурном смысле также учитывает и координирует набор изменений. Одна SqliteTransaction не является полным воплощением этого шаблона. В данной работе достаточно его понимания и простой транзакционной ReplaceAll, без отдельной системы отслеживания объектов.

3. Пример: список объектов в таблице

Этот пример проверяет только создание двух объектов и привязку таблицы. Он не содержит решения поиска, SQLite или восстановления. В

Visual Studio создайте отдельный небольшой проект Windows Forms App с именем BindingDemo. На Form1 разместите DataGridView с именем grid и BindingSource с именем source. У grid задайте Dock = Fill, ReadOnly = True и AllowUserToAddRows = False. AutoGenerateColumns оставьте True.

Добавьте публичный класс Pilot в том же пространстве имён BindingDemo. Код класса вводится отдельно от Form1, а не внутрь обработчика кнопки.

```
public class Pilot
{
    public string Name { get; set; } = "";
    public int LapSeconds { get; set; }
}
```

Откройте код Form1. Внутри конструктора после InitializeComponent поместите следующие команды. Список здесь локальный, поскольку после назначения он остаётся доступен источнику связи. Источник source уже создан визуальным конструктором, второй одноимённый объект не объявляйте.

```
List<Pilot> rows = new List<Pilot>
{
    new Pilot { Name = "Анна", LapSeconds = 94 },
    new Pilot { Name = "Борис", LapSeconds = 126 }
};
source.DataSource = rows;
grid.DataSource = source;
```

Выполните сборку и F5. Появятся два столбца Name и LapSeconds и две строки: Анна с числом 94 и Борис с числом 126. Значения времени являются данными примера. Если столбцы не появились, проверьте публичность свойств и AutoGenerateColumns. После проверки вернитесь к своему RaceLog. Отдельный запускаемый пример привязки также приложен к комплекту для сравнения с самостоятельно собранной версией.



Рисунок 1 – Привязка небольшого списка к таблице

4. Самостоятельное задание

Создайте законченную вторую версию RaceLog. Пользователь должен добавлять заезд, выбирать существующую запись, изменять её поля, удалять её после подтверждения, искать пилота и выбирать трассу. После закрытия и повторного запуска данные должны сохраниться. Сохранение копии включает весь журнал, даже если видны только результаты поиска. Восстановление заменяет весь журнал после проверки и подтверждения, а не дописывает строки.

Самостоятельно реализуйте RaceRules.Validate и ValidateImport, RaceQuery.Apply, все операции RaceRepository, методы RaceTransfer и обработчики формы. Модель RaceResult, значения учебного задания, примеры данных и открытые проверки предоставлены для единых имён. В запасном стартовом проекте методы помечены TODO и не содержат основного решения.

Модель и учебные данные

Свойство	**Тип**	**Значение**
Id	long	Ключ SQLite. При обычном добавлении назначается базой
Driver	string	Имя пилота после Trim, не пустое
Car	string	Street, Sport или Prototype
Track	string	Учебный овал, Городское кольцо или Спринт
LapSeconds	int	Целые секунды от 30 до 600 включительно

Данные ниже вымышлены для учебного приложения. Названия автомобилей являются вариантами выбора, а не характеристиками реальных моделей. Допустимое время от 30 до 600 секунд повторяет ЛР 1. Оно нужно

для тренировки проверки ввода и не является универсальным правилом автосимуляторов.

Id	Пилот	Автомобиль	Трасса	Время, с
1	Анна	Street	Учебный овал	94
2	Борис	Sport	Городское кольцо	126
3	Анна	Prototype	Спринт	73
4	Вера	Street	Учебный овал	101
5	Глеб	Sport	Спринт	80
6	Дина	Prototype	Городское кольцо	118

На промежуточном этапе используйте эти объекты в памяти. После реализации добавления внесите их через интерфейс. После реализации восстановления используйте основную копию «Заезды» из папки «Данные», чтобы снова получить точно Id 1-6. В том же наборе находятся одинаковая копия XML, «Ошибка времени», «Повтор Id», «Повреждённая копия» и «Пустой журнал». Их назначение объяснено на этапе 8.

5. Порядок выполнения

Этап 1. Подготовьте свою работу и подключите библиотеку

1.1. Откройте отдельную копию своей ЛР 1

Завершите запущенное приложение. В проводнике скопируйте всю папку своего решения ЛР 1 в отдельную папку ЛР 2. Откройте Visual Studio 2026 через меню «Пуск», выберите «Открыть проект или решение» (Open a project or solution) и откройте решение из новой папки. В Solution Explorer выделите RaceLog. Проверьте, что исходная форма первой работы открывается в конструкторе и запускается по F5. Не редактируйте единственный сохранённый экземпляр ЛР 1.

1.2. Определите, откуда начинаете

Основной путь выполнения начинается с вашей работающей формы ЛР 1. В комплект также входит запасной стартовый проект с пустой формой, классом RaceResult, обозначенными методами и открытыми проверками. Он не содержит готового интерфейса или основного решения. Используйте его только как чистую структуру или источник имён и данных. Не заменяйте свою рабочую форму пустой. Не копируйте заглушки поверх уже написанных методов. В исходной заготовке отмеченные методы специально не реализованы.

1.3. Установите Microsoft.Data.Sqlite

В Visual Studio выберите проект RaceLog, затем «Проект», «Управление пакетами NuGet» (Project, Manage NuGet Packages). На вкладке «Обзор» (Browse) установите источник nuget.org и найдите Microsoft.Data.Sqlite. Проверьте издателя Microsoft. Отключите Include prerelease и выберите стабильную версию 10.0.11. Нажмите Install и подтвердите предложенные зависимости. Не выбирайте Microsoft.Data.Sqlite.Core вместо полного пакета. Для JSON и XML отдельные библиотеки устанавливать не требуется.

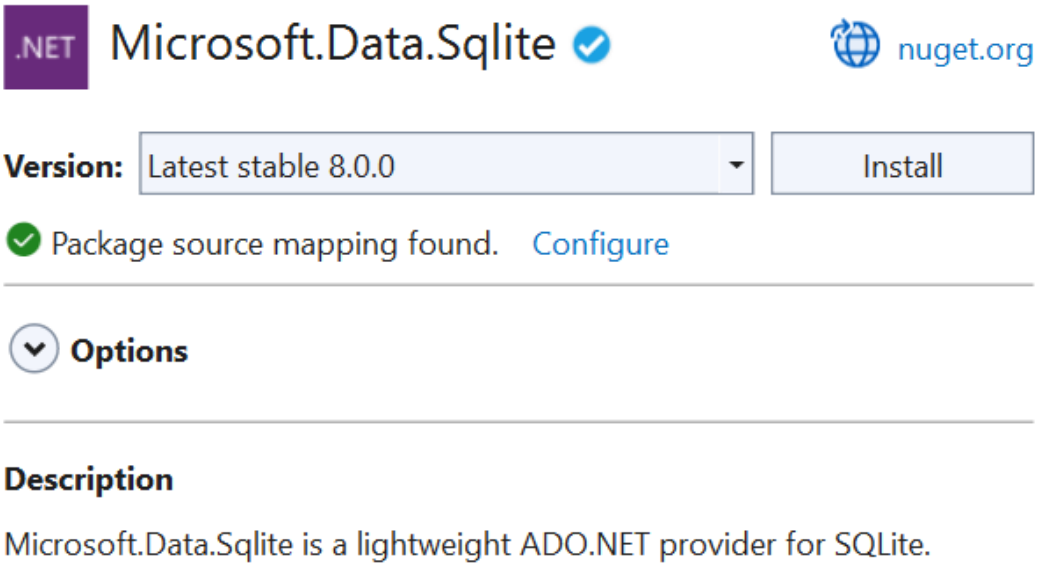


Рисунок 2 – Карточка Microsoft.Data.Sqlite в менеджере пакетов

На снимке показана версия 8.0.0. Для ЛР 2 выберите 10.0.11, указанную в условии. Нужен полный пакет Microsoft.Data.Sqlite, не похожее название.

1.4. Проверьте результат установки

В Solution Explorer разверните Dependencies, затем Packages. Найдите Microsoft.Data.Sqlite. Выполните «Сборка», «Собрать решение» (Build, Build Solution). Если библиотека не найдена, проверьте источник пакетов и соединение. Если шаблон не поддерживает .NET 10, сравните свойства проекта с ЛР 1. Не переходите на .NET Framework ради обхода ошибки. После успешной сборки закройте запущенную форму перед дальнейшими изменениями.

Что должно получиться

Сохранена отдельная копия работы. Проект собирается с выбранной зависимостью. Прежние действия формы ещё работают. Пока установка библиотеки не создала ни таблицу SQLite, ни постоянное хранение.

Этап 2. Создайте модель записи и общие правила

2.1. Добавьте класс RaceResult

В обозревателе решений щёлкните проект правой кнопкой, выберите «Добавить», «Класс» (Add, Class) и назовите класс RaceResult. Поместите его в пространство имён RaceLog. Строка namespace RaceLog; относит последующие объявления к этому пространству. Она не должна находиться внутри класса Form1. Задайте пять свойств по таблице модели. Не помещайте элементы управления внутрь RaceResult: модель описывает заезд, а не окно.

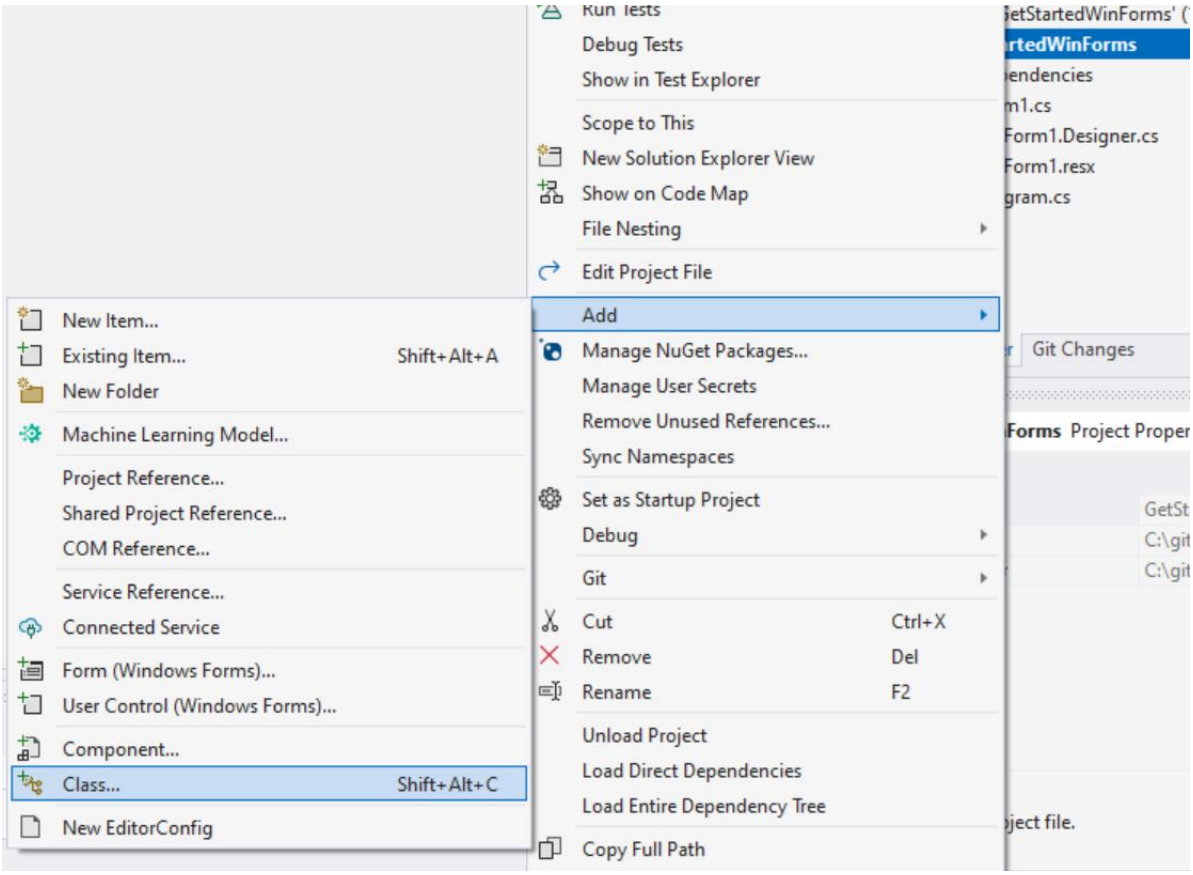


Рисунок 3 – Команда Add, Class в Visual Studio

На снимке другой проект. В своей работе выделите RaceLog и создайте класс RaceResult. Форма и обработчик события для этого не выбираются.

```
public class RaceResult
{
    public long Id { get; set; }
    public string Driver { get; set; } = "";
    public string Car { get; set; } = "";
    public string Track { get; set; } = "";
    public int LapSeconds { get; set; }
}
```

2.2. Создайте общую проверку

Добавьте класс RaceRules. Объявите нижнюю границу MinLapSeconds = 30 и верхнюю MaxLapSeconds = 600. Эти числа означают половину минуты и десять минут учебного круга. Сохраните варианты автомобилей и трасс из ЛР 1. В методе Validate(RaceResult row) проверьте непустое имя, известные значения выбора и допустимость времени. При нарушении выбрасывайте ArgumentException с конкретным текстом. Самостоятельно реализуйте все проверки. Они понадобятся и форме, и чтению копии.

2.3. Отличайте идентификатор от позиции

При создании нового объекта Id можно оставить равным нулю до вставки. SQLite назначит его при добавлении. Для существующих и восстановленных записей Id должен быть положительным. Не вычисляйте его как номер выделенной строки или Items.Count. Одинаковые пилот, автомобиль, трасса и время разрешены: это могут быть разные заезды. Запрещены только два одинаковых Id в одной восстанавливаемой копии.

2.4. Подготовьте шесть объектов в памяти

Перепишите шесть строк учебных данных в инициализатор List<RaceResult>. Каждый объект создаётся через new. Добавьте поле allResults в класс формы, но вне её методов. На этом промежуточном этапе в нём находятся учебные объекты, а не результаты запроса. После подключения SQLite на этапе 5 источником станет GetAll. Не вставляйте этот учебный список в базу автоматически при каждом запуске.

Что должно получиться

Проект собирается. У каждого объекта пять именованных свойств. В allResults находятся шесть учебных записей с Id от 1 до 6. Время 30 и 600 проходит проверку, 29 и 601 отклоняется. Если возникает ошибка компиляции, сначала проверьте, что класс и метод не вставлены внутрь другого метода.

Этап 3. Замените список таблицей и настройте привязку

3.1. Откройте конструктор

В Solution Explorer выберите Form1, вызовите «Открыть конструктор» (View Designer). Сохраните поля txtDriver, cmbCar, cmbTrack и существующие кнопки первой работы. Поле времени, названное в ЛР 1 txtTime, переименуйте через (Name) в txtSeconds и обновите обращения к нему в своём коде. Удалите ListBox lstResults и добавьте DataGridView с именем gridResults. После удаления прежний код с lstResults нужно заменить новым порядком работы, а

не оставлять ссылки на отсутствующий элемент. Форма должна показывать таблицу и нижние команды целиком при выбранном минимальном размере.

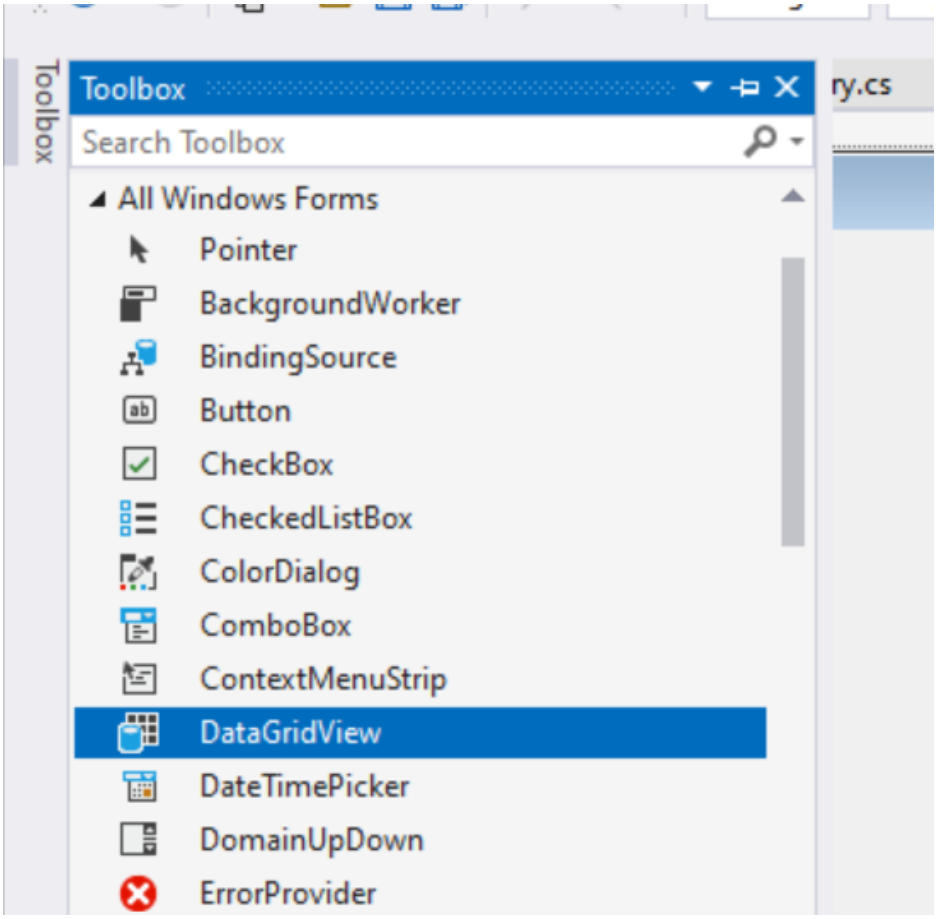


Рисунок 4 – Элементы DataGridView и BindingSource в Toolbox

Перетащите DataGridView на форму и задайте имя gridResults. BindingSource находится в том же списке, но при добавлении появляется в области компонентов под формой.

3.2. Настройте столбцы

Выделите gridResults. В Properties задайте ReadOnly = True, AllowUserToAddRows = False, MultiSelect = False, SelectionMode = FullRowSelect. Через свойство Columns откройте редактор столбцов. Добавьте пять текстовых столбцов по таблице ниже. У каждого укажите Name, HeaderText и DataPropertyName. Для всех столбцов задайте SortMode = NotSortable. Для таблицы можно выбрать AutoSizeColumnsMode = Fill и Anchor по всем четырём сторонам. Последнее сохраняет её размер относительно краёв при изменении окна. AutoGenerateColumns не

отображается в Properties. В конструкторе после InitializeComponent и до назначения DataSource добавьте команду gridResults.AutoGenerateColumns = false. Она отключает автоматическое создание столбцов, поскольку вы задаёте их вручную.

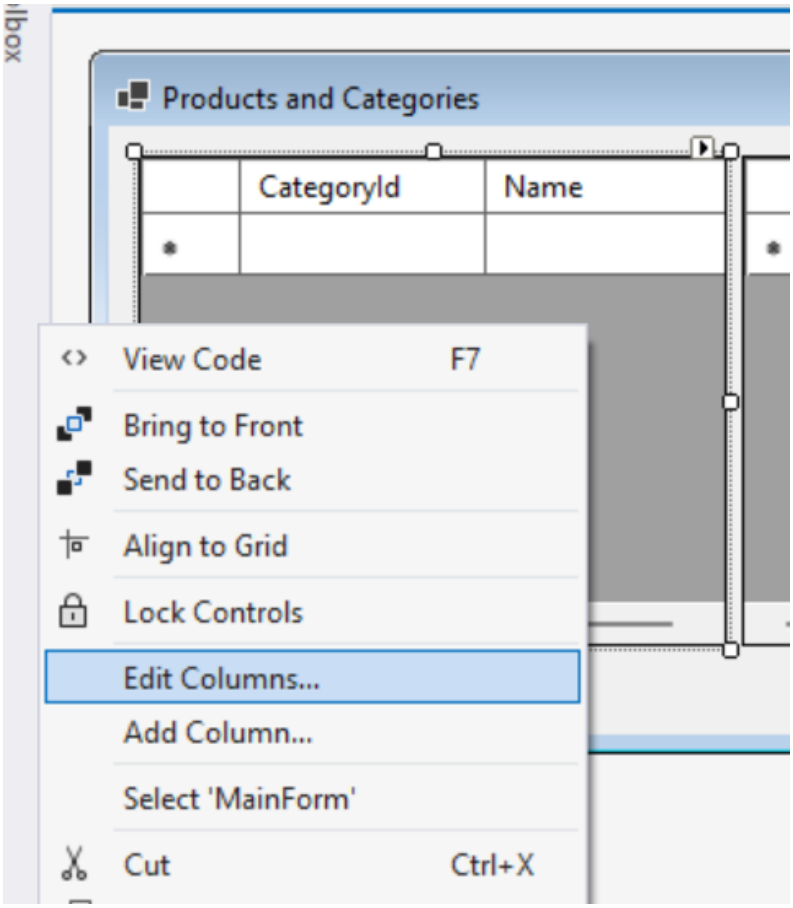


Рисунок 5 – Открытие редактора столбцов из контекстного меню

Альтернативный путь к той же настройке: щёлкните таблицу правой кнопкой и выберите Edit Columns. Столбцы чужого примера копировать не нужно.

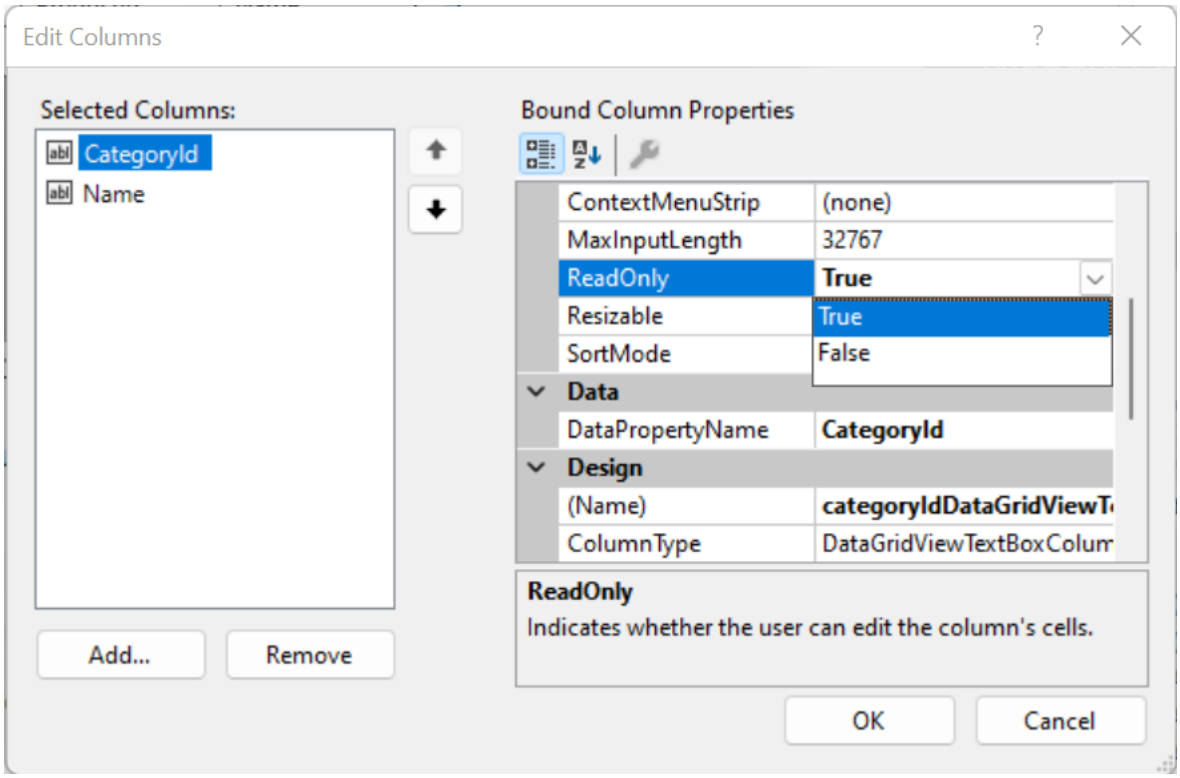


Рисунок 6 – Свойства столбца в редакторе Edit Columns

Слева выбирается столбец, справа меняются его свойства. CategoryId на снимке заменяется нужным свойством вашей модели. Для столбца пилота укажите Driver в DataPropertyName и «Пилот» в HeaderText.

Name	**HeaderText**	**DataPropertyName**
colId	Id	Id
colDriver	Пилот	Driver
colCar	Автомобиль	Car
colTrack	Трасса	Track
colLapSeconds	Время, с	LapSeconds

3.3. Добавьте BindingSource

Компонент BindingSource можно перетащить из Toolbox. Он появится в области компонентов под формой, поскольку отдельного видимого окна у него нет. Назовите его source. Другой вариант состоит в объявлении поля BindingSource source = new BindingSource() в коде формы. Выберите один вариант, не создавайте одновременно два одноимённых поля. В конструкторе после InitializeComponent назначьте source.DataSource = allResults и

gridResults.DataSource = source. При использовании поля в коде освободите его вместе с формой.

3.4. Проверьте небольшой пример

Сначала выполните описанный выше пример с двумя пилотами в отдельном маленьком проекте либо на временной форме. Затем вернитесь в RaceLog. Запустите приложение. Должны появиться шесть строк и пять столбцов с русскими заголовками. Если один столбец пуст, сравните DataPropertyName с именем свойства. Если не появилось ни одной строки, проверьте порядок присваивания источников и наличие объектов в allResults. Не пытайтесь одновременно вызывать Rows.Add у уже связанной таблицы.

Что должно получиться

В таблице видны все шесть записей. Добавлять строку через пустой последний ряд и править ячейку напрямую нельзя. Выбор выделяет одну строку целиком. На этом этапе таблица работает без базы, поэтому ошибки привязки можно исправить отдельно от SQL.

Этап 4. Реализуйте поиск и фильтр

4.1. Добавьте элементы поиска

Над таблицей разместите txtSearch, выпадающий список cmbTrackFilter, кнопки btnFilter «Найти» и btnResetFilter «Сброс поиска». У списка установите DropDownStyle = DropDownList. Его первый вариант называется «Все трассы», остальные точно повторяют названия трасс задания. Начальное SelectedIndex равно нулю, то есть выбран первый вариант. Это относится к списку фильтра, а не к идентификаторам заездов.

4.2. Напишите RaceQuery.Apply

Добавьте отдельный класс RaceQuery со статическим методом Apply(List<RaceResult> all, string text, string track). Внутри удалите пробелы по краям запроса. Отберите записи, имя которых содержит запрос без учёта регистра. Затем оставьте выбранную трассу или все записи, если выбран вариант «Все трассы». Упорядочьте по Id и вызовите ToList. Верните новый

список. Исходный all не изменяйте. Не используйте BindingSource.Filter для обычного List<RaceResult>.

4.3. Свяжите кнопки с кодом

Выделите btnFilter, откройте события в Properties значком молнии и создайте обработчик Click. Пусть он вызывает отдельный метод ApplyFilter формы. В этом методе результат RaceQuery.Apply назначается source.DataSource. Затем выполните gridResults.ClearSelection и обновите lblStatus: «Показано: … Всего: …». Количество видимых строк берите из source.Count, полное количество из allResults.Count. Кнопка сброса очищает txtSearch, выбирает «Все трассы» и вызывает тот же ApplyFilter.

4.4. Проверьте совместное действие условий

Запрос АННА без ограничения трассы должен показать Id 1 и 3. При выборе Спринт должен остаться Id 3. Запрос «нет такого» даёт ноль совпадений при общем количестве шесть. Нажмите «Сброс поиска»: должны вернуться все шесть строк. Если возвращаются не все записи, проверьте, не присвоили ли вы отфильтрованный список полю allResults.

Что должно получиться

Поиск и трасса работают вместе. Скрытые строки не удаляются. После изменения представления нет случайно выделенной записи. Пользователь видит оба количества и может одним действием вернуться ко всему журналу.

Этап 5. Создайте SQLite-хранилище и чтение

5.1. Определите путь

В конструкторе формы получите Environment.SpecialFolder.LocalApplicationData, добавьте подпапку RaceLogLab2 через Path.Combine и создайте её Directory.CreateDirectory. Внутри используйте постоянное имя хранилища из примера кода. Полный путь покажите в отдельной метке lblStorage. Не сохраняйте базу по случайному пути из чужого снимка и не считайте папку сборки постоянным

местом хранения. На одном учебном компьютере проверяйте, что открыли именно свой журнал.

5.2. Добавьте RaceRepository

Добавьте класс с конструктором, принимающим путь. В конструкторе соберите строку подключения через SqliteConnectionStringBuilder и сохраните её в поле. В Initialize откройте SqliteConnection, создайте команду CREATE TABLE IF NOT EXISTS для RaceResults и выполните ExecuteNonQuery. Схема приведена ниже. Конструкция @ перед строкой C# разрешает записывать SQL на нескольких строках. Она не означает выполнение SQL: запрос выполняется только при вызове метода команды.

```
CREATE TABLE IF NOT EXISTS RaceResults (
    Id INTEGER PRIMARY KEY,
    Driver TEXT NOT NULL,
    Car TEXT NOT NULL,
    Track TEXT NOT NULL,
    LapSeconds INTEGER NOT NULL
        CHECK(LapSeconds BETWEEN 30 AND 600)
);
```

5.3. Реализуйте GetAll

Создайте пустой List<RaceResult>. На открытом соединении выполните SELECT Id, Driver, Car, Track, LapSeconds FROM RaceResults ORDER BY Id. Используйте ExecuteReader и цикл while (reader.Read()). Номера от 0 до 4 соответствуют этому порядку столбцов. Для Id вызовите GetInt64, для строк GetString, для времени GetInt32. Создавайте новый RaceResult для каждой строки и добавляйте его в список. Верните список после цикла. При ошибке не возвращайте пустой список как будто чтение прошло успешно.

5.4. Подключите чтение к форме

В конструкторе после создания элементов вызовите Initialize, затем Reload. Метод Reload формы присваивает результат repository.GetAll полю allResults и вызывает ApplyFilter. Добавьте кнопку btnReload «Обновить» с тем же вызовом. Обрабатывайте ошибки доступа через сообщение. При самом первом запуске новой базы строк нет. Это ожидаемо. Шесть учебных строк вы

внесёте на следующем этапе или восстановите позже после реализации импорта. Не заполняйте базу начальными данными при каждом запуске.

Что должно получиться

Форма показывает пустой новый журнал и полный путь к базе. Повторная подготовка не стирает данные. GetAll умеет возвращать существующий пустой список. SQLite и привязка соединены через объекты, а SQL находится в репозитории.

Этап 6. Реализуйте добавление, изменение и удаление

6.1. Напишите чтение полей

В форме создайте метод ReadInput(long id), который возвращает новый RaceResult. Прочитайте txtDriver.Text.Trim, значения cmbCar и cmbTrack. Преобразуйте txtSeconds через int.TryParse. При неудаче покажите понятную причину. Передайте полученный объект в RaceRules.Validate. Метод не должен менять существующий объект выбранной строки и не должен выполнять SQL. Для добавления передайте id = 0, для изменения сохраните Id выбранной записи.

6.2. Реализуйте Add

В RaceRepository.Add проверьте данные, откройте соединение и выполните параметризованный INSERT без указания Id. Передайте четыре значения через Parameters.AddWithValue. После успешной вставки на том же соединении выполните SELECT last_insert_rowid() и верните long. В обработчике btnAdd сначала получите объект ReadInput(0), затем вызовите Add, после успеха Reload и очистку полей. При ошибке проверки ни одна новая строка не появляется, а введённые данные остаются для исправления.

6.3. Подключите выбор строки

Создайте у gridResults обработчик SelectionChanged через окно событий. Если выбрана одна строка, получите её DataBoundItem as RaceResult и проверьте null. Скопируйте значения в верхние поля. Индекс SelectedRows[0] означает первую выделенную строку, а не запись с Id 0. Если выделение

отсутствует, обработчик просто заканчивается. Не используйте source.Current как доказательство выделения: текущий объект может оставаться после ClearSelection.

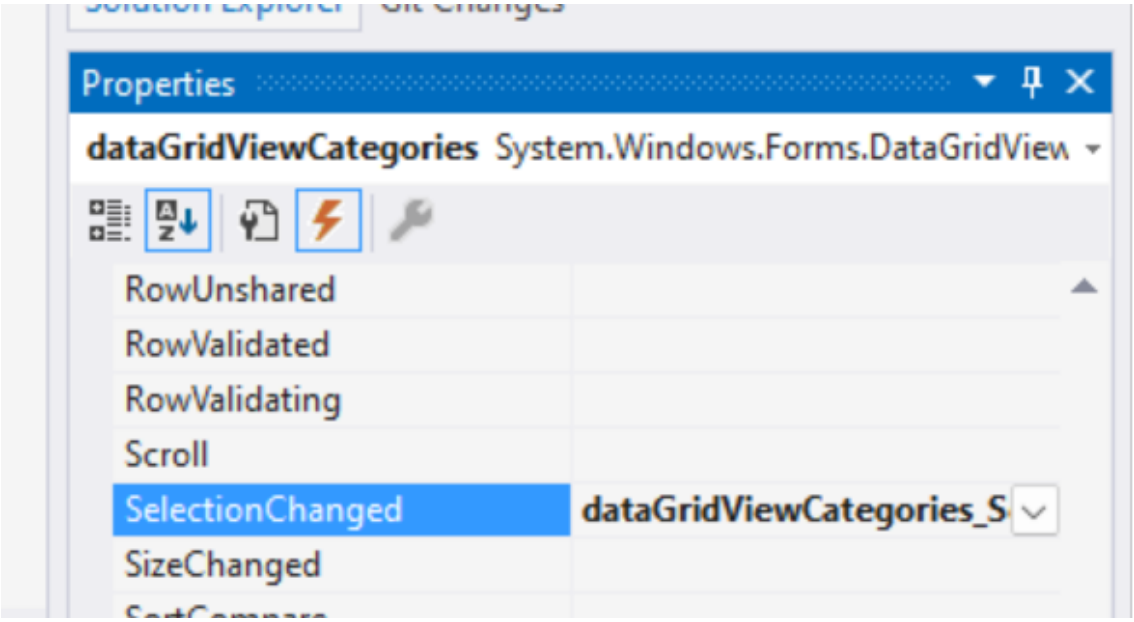


Рисунок 7 – Событие SelectionChanged в Properties

Выделите именно gridResults. Значок молнии открывает события. Двойной щелчок по полю справа от SelectionChanged создаёт обработчик. Имя dataGridViewCategories на снимке относится к примеру Microsoft.

6.4. Реализуйте правку

Добавьте btnUpdate «Сохранить правку». В её обработчике сначала проверьте выделение, получите Id выбранного объекта, затем создайте новый объект через ReadInput(id). В RaceRepository.Update выполните UPDATE с WHERE Id=$id. Старый Id не меняется. Проверьте, что ExecuteNonQuery затронул одну строку. После успеха вызовите Reload и очистите поля. Если новое значение больше не подходит текущему поиску, строка исчезнет из представления, но останется в базе.

6.5. Реализуйте удаление и очистку

В btnDelete сначала проверьте выбор и спросите подтверждение с указанием Id и пилота. При ответе «Нет» завершите обработчик. При согласии вызовите Delete(id), где команда содержит WHERE Id=$id и параметр. После успеха перечитайте таблицу. BtnClear очищает только поля и выделение, но не

журнал. После каждой операции сохраните правило: сначала успешное изменение SQLite, потом новый список и сообщение.

6.6. Проверьте сохранение между запусками

Введите шесть учебных записей по таблице, каждый раз нажимая «Добавить». Затем измените время Анны на Спринте с 73 на 72 через фильтр. Убедитесь, что Id этой записи не поменялся и другие строки остались прежними. Закройте приложение и запустите снова. Правка должна сохраниться. Для дальнейших общих проверок верните время 73. Если добавляли и удаляли пробные строки, Id могут отличаться от 1-6 до восстановления основной учебной копии. Сравнивайте выбранный объект, а не предполагайте конкретный Id после произвольной истории добавлений.

Что должно получиться

Работают все четыре операции. Выбор после фильтрации не приводит к изменению соседнего заезда. Закрытие и повторный запуск сохраняют данные. Пустое имя, недопустимое время и отсутствие выбора не приводят к ложным сообщениям об успехе.

Этап 7. Сохраните копии журнала

7.1. Создайте RaceTransfer

Добавьте статический класс с методами SaveJson и SaveXml, которые получают путь и List<RaceResult>. Они не должны обращаться к полям формы, открывать диалоги или выполнять SQL. Добавьте using System.Text.Json для JSON и using System.Xml.Serialization для XML. Оба средства входят в выбранную платформу. Не устанавливайте дополнительные библиотеки сериализации.

7.2. Реализуйте сохранение JSON

Создайте JsonSerializerOptions с WriteIndented = true, получите текст через JsonSerializer.Serialize и запишите его File.WriteAllText. WriteIndented нужен только для удобного чтения. Последовательности Unicode вместо кириллицы при стандартных настройках допустимы, поскольку обратно

читаются те же буквы. Не добавляйте к числу секунд суффикс «с» внутри свойства. LapSeconds сохраняется как число.

7.3. Реализуйте сохранение XML

Создайте XmlSerializer для typeof(List<RaceResult>). Откройте поток File.Create с using и передайте поток и список в Serialize. Не записывайте разметку вручную склеиванием строк. Сериализатор сам обрабатывает специальные символы. Убедитесь, что RaceResult является публичным классом с публичными get и set. Если собственный конструктор класса требует параметры, добавьте доступный конструктор без параметров.

7.4. Подключите диалоги

Добавьте кнопки btnSaveJson «В JSON» и btnSaveXml «В XML». Каждая открывает SaveFileDialog с понятным названием и подходящим фильтром. Если ShowDialog вернул не DialogResult.OK, действие заканчивается без записи. После выбора пути получите весь журнал через repository.GetAll и передайте его в нужный метод RaceTransfer. Не экспортируйте только source или Rows. Выбирайте отдельную копию, не рабочую базу. При совпадении выбранного пути с путём SQLite откажитесь от сохранения, чтобы не перезаписать хранилище.

Что должно получиться

При активном поиске «Анна» каждая копия всё равно содержит весь журнал. Обычный экспорт не меняет базу. Отмена диалога ничего не записывает. При ошибке доступа нет сообщения «Сохранено», а рабочие данные остаются в SQLite.

Этап 8. Восстановите журнал без потери прежних данных

8.1. Читайте в отдельный список

Реализуйте LoadJson и LoadXml в RaceTransfer. Для JSON используйте File.ReadAllText и JsonSerializer.Deserialize<List<RaceResult>>. Проверьте null. Для XML создайте XmlReader и XmlSerializer, прочитайте объект и примените as List<RaceResult>, затем проверьте null. В настройках XmlReader

запретите DTD и внешние ресурсы. Это готовая настройка чтения локальных учебных данных, а не отдельная тема лабораторной. Ничего не удаляйте из базы во время чтения.

```
XmlReaderSettings settings = new XmlReaderSettings
{
    DtdProcessing = DtdProcessing.Prohibit,
    XmlResolver = null
};
using XmlReader reader = XmlReader.Create(path, settings);
```

8.2. Проверьте всю копию

Напишите RaceRules.ValidateImport. Для каждого объекта вызовите Validate, проверьте положительность Id и отсутствие уже встретившегося Id. Для запоминания прочитанных идентификаторов достаточно List<long> и Contains. Неверная запись в конце списка отклоняет весь импорт. Корректный пустой список разрешён. Null вместо списка, пустой объект, неизвестный автомобиль и время 601 являются ошибками.

8.3. Напишите ReplaceAll

В репозитории после полной проверки откройте одно соединение и одну SqliteTransaction через BeginTransaction. Создайте DELETE FROM RaceResults без WHERE, потому что здесь намеренно заменяется весь журнал. Назначьте команде Transaction. Затем в цикле вставьте каждую запись вместе с её Id. Все команды создаются на том же соединении и получают ту же Transaction. Не вызывайте обычный Add, если он открывает другое соединение. Commit вызывается ровно после последней успешной вставки. При исключении не подавляйте ошибку. Using освободит незавершённую транзакцию с откатом.

8.4. Свяжите восстановление с интерфейсом

Добавьте btnLoadJson «Из JSON» и btnLoadXml «Из XML». Откройте OpenFileDialog и завершите действие при отмене. Прочитайте копию и проверьте её. Покажите предупреждение: весь текущий журнал будет заменён. Укажите число нынешних и новых записей. Только после ответа «Да» вызывайте ReplaceAll. После успеха очистите поиск, выберите «Все трассы»,

вызовите Reload и снимите выделение. Повторное восстановление одной копии не должно удваивать строки.

8.5. Проверьте учебные копии

Восстановите основную копию «Заезды» и проверьте Id 1-6. Затем по очереди попробуйте «Ошибка времени», «Повтор Id» и «Повреждённая копия». После каждого отказа основной журнал должен остаться прежним. «Пустой журнал» является корректной пустой копией: сначала отмените замену и убедитесь в сохранности строк. Затем подтвердите замену, проверьте пустой журнал и восстановите основную копию снова.

Что должно получиться

Оба представления восстанавливают одинаковые объекты и Id. Неверная копия не разрушает существующий журнал. Пользователь управляет заменой через подтверждение. Пустая копия обрабатывается отдельно от ошибки чтения.

Этап 9. Проверьте приложение и подготовьте защиту

9.1. Выполните пользовательские проверки

Сначала восстановите основную копию из шести записей. Далее пройдите все сценарии раздела «Проверка результата». После проверок, которые меняют состав журнала, восстановите основную копию перед следующим независимым сценарием. Записывайте не только наличие сообщения, но и фактические Id, значения и количество строк.

9.2. Выполните сборку и открытые проверки

Откройте встроенный терминал Visual Studio через «Вид», «Терминал». Перейдите в корневую папку стартового или преподавательского комплекта, где находятся папки RaceLog и «Проверки». Команды ниже собирают приложение и запускают проверки данных. Проверки используют отдельную временную базу, а не ваш журнал. До реализации заглушек часть проверок ожидаемо не проходит. Не считайте успешную компиляцию доказательством правильного поведения кнопок.

```
dotnet build RaceLog
dotnet run --project Проверки
```

9.3. Разберите ошибки по месту возникновения

Если не видна таблица, проверьте объекты и привязку. Если пуст только один столбец, сравните DataPropertyName. Если ошибка возникает при сохранении, проверьте параметры SQL и путь. Если программа меняет не тот заезд, сравните Id и позицию. Если после неудачного импорта пропали данные, найдите преждевременный DELETE или команды вне транзакции. Исправьте причину и повторите сценарий, который обнаружил ошибку.

Что должно получиться

Проект собирается на вашем Windows-компьютере и выполняет все обязательные действия в интерфейсе. В отчёте есть конкретные примеры и результаты проверок. Вы можете объяснить путь данных между формой, объектами, репозиторием и SQLite.

6. Ожидаемый вид приложения

Сверху расположите поля пилота, автомобиля, трассы и времени. Ниже отдельную строку поиска и фильтра. Центральную часть отдайте таблице. Под ней разместите команды изменения и обмена данными. Ещё ниже должны помещаться сообщение о результате и полный путь к базе. Не обязательно повторять координаты эталона. Важны понятные подписи, ровное выравнивание, читаемые строки и доступность всех кнопок.

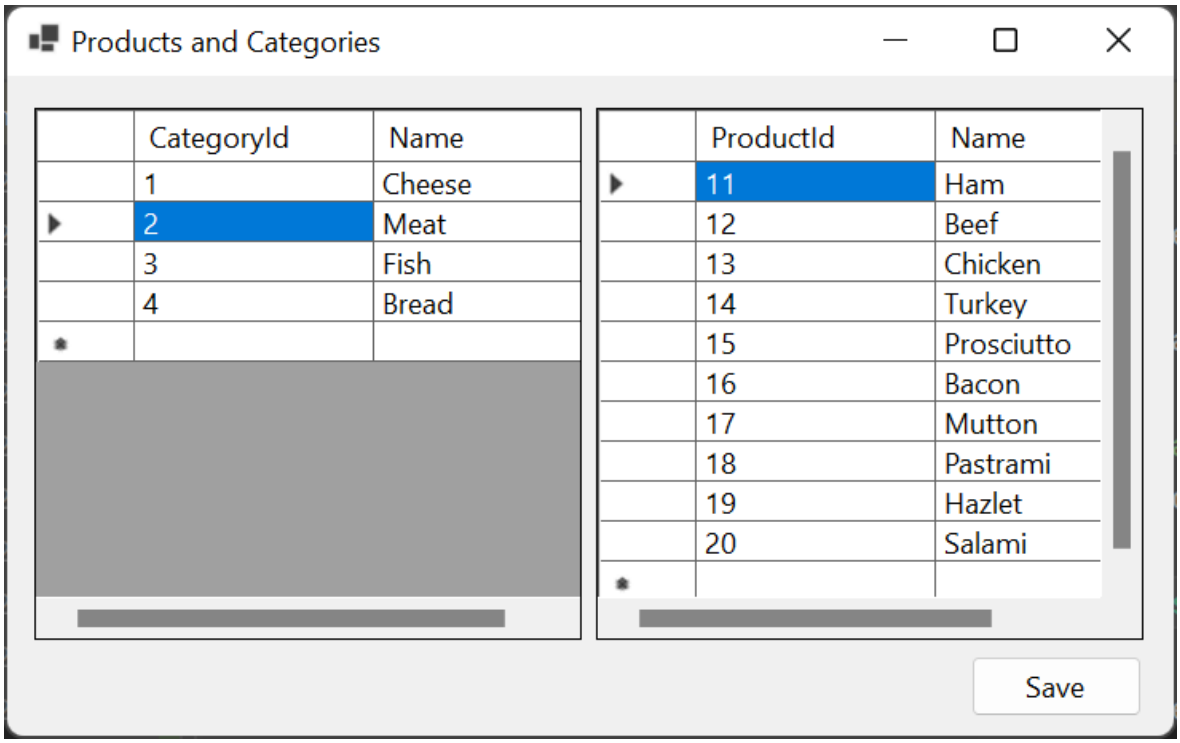


Рисунок 8 – Работающие DataGridView в примере

Показано приложение Products and Categories, не RaceLog. На снимке видны строки, заголовки и выделение. В вашей форме одна основная таблица, а правка выполняется через поля и кнопки по заданию.

Элемент	**Имя**	**Действие**
TextBox	txtDriver, txtSeconds	Ввод пилота и времени
ComboBox	cmbCar, cmbTrack	Выбор из заданных вариантов
TextBox, ComboBox	txtSearch, cmbTrackFilter	Поиск по пилоту и ограничение трассы
DataGridView	gridResults	Показ и выделение записей
BindingSource	source	Связь объектов с таблицей
Button	btnAdd, btnUpdate, btnDelete	Добавление, правка, удаление
Button	btnClear, btnReload	Очистка ввода, чтение базы
Button	btnFilter, btnResetFilter	Применение и сброс поиска

Элемент	Имя	Действие
Button	btnSaveJson, btnLoadJson	Копия JSON и восстановление
Button	btnSaveXml, btnLoadXml	Копия XML и восстановление
Label	lblStatus, lblStorage	Результат действия и путь к базе

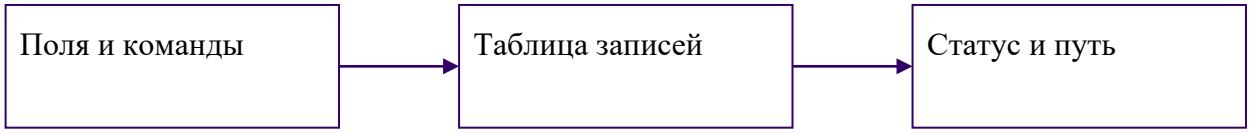


Рисунок 9 – Схема основных областей формы

Для схемы проверки восстановите основную копию и выполните поиск анна. При варианте «Все трассы» видны Id 1 и 3, а статус показывает «Показано: 2. Всего: 6». При выборе Спринт остаётся Id 3. После сброса снова видны все шесть. Во всех случаях кнопки копирования работают со всем журналом. Две версии копии содержат одинаковые сведения.

7. Проверка результата

Проверка 1. Начальное чтение

После восстановления основной копии видны шесть строк с Id 1-6.

Проверка 2. Поиск без регистра

АННА и анна дают Id 1 и 3.

Проверка 3. Два условия

Поиск анна и трасса Спринт дают только Id 3.

Проверка 4. Нет совпадений

Ноль показанных строк, всего шесть. Сброс возвращает все шесть.

Проверка 5. Корректное добавление

Егор, Street, Учебный овал, 95 добавляется с назначенным Id.

Проверка 6. Пустое имя

Пустая строка и одни пробелы отклоняются. Количество не меняется.

Проверка 7. Число секунд

Пустое время, быстро, 94.5 и 94,5 отклоняются как нецелое время.

Проверка 8. Нижний край

29 отклоняется, 30 принимается.

Проверка 9. Верхний край

600 принимается, 601 отклоняется.

Проверка 10. Апостроф

Имя О'Нил сохраняется буквально, запрос не ломается.

Проверка 11. Правка после фильтра

У Id 3 измените 73 на 72. Id 1 не меняется, Id 3 сохраняется.

Проверка 12. Неверная правка

Для выбранной записи введите 601. Прежнее время остаётся в базе.

Проверка 13. Нет выбора

Правка и удаление без выделения просят выбрать строку.

Проверка 14. Отмена удаления

Ответ Нет не меняет журнал.

Проверка 15. Удаление после фильтра

Удаляется ровно выбранный Id, не ряд с похожей позицией.

Проверка 16. Очистка ввода

Поля и выделение очищены, записи не удалены.

Проверка 17. Перезапуск

После закрытия и повторного запуска последние изменения сохранились.

Проверка 18. Экспорт с фильтром

Поиск показывает две строки, копия содержит все шесть.

Проверка 19. Восстановление JSON

Поля и Id совпадают с основной копией.

Проверка 20. Восстановление XML

Получены те же шесть объектов и Id.

Проверка 21. Повторное восстановление

Остаются шесть строк, а не двенадцать.

Проверка 22. Неверные копии

Время 601, повтор Id и нечитаемая копия отклоняются, прежние данные сохранены.

Проверка 23. Отмена замены

После чтения правильной копии ответ Нет не меняет журнал.

Проверка 24. Пустая копия

После согласия журнал пуст. После отмены прежние строки остаются.

Проверка 25. Недоступная запись

Попытка сохранения в недоступное место даёт сообщение, не ложный успех.

Открытые программные проверки содержат 33 сценария для модели, поиска, SQLite, копий и отката. Они не запускают форму и не нажимают её кнопки. Их выполнение дополняет, но не заменяет приведённые пользовательские проверки.

8. Что показать в отчёте и на защите

В начале отчёта укажите цель и использованные средства. Приведите собственную схему связи формы, модели и репозитория. Покажите снимок таблицы из шести записей, результат поиска, пример успешной правки с сохранённым Id и сообщение при неверном вводе. Снимки должны быть получены из вашего работающего приложения, а не из схемы условия.

Поясните небольшой фрагмент фильтрации, параметризованный UPDATE и транзакционную ReplaceAll. Для JSON и XML покажите соответствие одной записи модели. В результатах проверок зафиксируйте вход, ожидаемое и фактическое поведение, особенно для фильтрации,

перезапуска и отказа восстановления. Полный код не требуется дублировать на десятках страниц, передайте всю папку решения вместе с отчётом.

На защите откройте свой проект, соберите и запустите приложение. Восстановите основную копию, найдите Анну на Спринте, измените время, покажите неизменный Id. Затем покажите добавление, отказ неверному времени, удаление с отменой и подтверждением, перезапуск, экспорт при активном фильтре и отказ неверной копии без потери данных. По просьбе преподавателя объясните выбранный метод или исправьте небольшую ошибку в нём.

9. Контрольные вопросы

1. Почему RaceResult удобнее строки с разделителями?
2. Что делают get и set?
3. Почему присваивание объекта другой переменной не создаёт независимую копию?
4. Чем отличается List<RaceResult> от string[]?
5. Что делает BindingSource и чего он не делает?
6. Почему DataPropertyName нельзя заменить русским заголовком?
7. Чем Id отличается от индекса строки?
8. Зачем хранить allResults отдельно?
9. Что означают r =>.., Where и ToList?
10. Почему BindingSource.Filter не подходит для обычного List<RaceResult>?
11. Зачем использовать параметры SQL?
12. Почему нельзя обновить таблицу до успешного UPDATE?
13. Что именно экспортирует приложение?
14. Чем корректный пустой список отличается от null и ошибки чтения?
15. Зачем транзакция, если все записи уже проверены?
16. Почему внутри ReplaceAll нельзя вызвать Add с новым соединением?
17. Чем Repository отличается от Unit of Work?
18. Какие ограничения остались у приложения?